

Izrada rješenja za nadzor komunikacije lokalne mreže
sa naglaskom na izlazne tokove podataka iz mreže
Dnevne bilješke o radu na diplomskom zadatku

Ante Čavar

Rok predaje rada: 15. lipnja 2026.

Sadržaj

Tjedan 12 (16. svibnja – 22. svibnja)	3
Tjedan 11 (9. svibnja – 15. svibnja)	8
Tjedan 10 (2. svibnja – 8. svibnja 2026.)	13
Tjedan 9 (25. travnja – 1. svibnja 2026.)	18
Tjedan 8 (18. travnja – 24. travnja 2026.)	23
Tjedan 7 (11. travnja – 17. travnja 2026.)	28
Tjedan 6 (4. travnja – 10. travnja 2026.)	33
Tjedan 5 (28. ožujka – 3. travnja 2026.)	37
Tjedan 4 (21. ožujka – 27. ožujka 2026.)	41
Tjedan 3 (14. ožujka – 20. ožujka 2026.)	45
Tjedan 2 (7. ožujka – 13. ožujka 2026.)	49
Tjedan 1 (27. veljače – 6. ožujka 2026.)	53

Tjedan 12

(16. svibnja – 22. svibnja)

Petak, 22. svibnja

Ukratko

- Pisanje poglavlja o evaluaciji sustava
- Definiranje metrika za usporedbu

Fokusirao sam se na poglavlje evaluacije gdje uspoređujem InfluxDB i Elasticsearch implementacije. Morao sam definirati jasne metrike za usporedbu: storage efficiency (koliko prostora zauzimaju isti podaci), query latency (brzina izvršavanja tipičnih upita), memory footprint (RAM potrošnja u idle i pod opterećenjem), i ease of deployment (koliko koraka treba za setup).

Pišući evaluaciju, shvatio sam da mi nedostaju konkretni brojevi. Za sada imam samo rough estimates iz prethodnih testiranja. Trebat ću napraviti formalne benchmark testove s istim datasetom na oba sustava kako bi imao vjerodostojne rezultate za diplomski.

Također sam pisao o false positive reduction mehanizmu. Objašnjavam koncept centralnog repozitorija, kako bi crowd-sourced heuristika funkcionirala, i kako bi opt-in/opt-out sustav bio implementiran s naglaskom na privatnost. Ovo je trenutno teorijski dio - implementacija dolazi kasnije.

Dio o target audience-u (male firme, privatni korisnici) zahtijevao je objašnjenje zašto postojeća enterprise rješenja nisu primjenjiva. Argumenti: visoka cijena (Splunk, QRadar), kompleksnost konfiguracije (zahtijevaju dedikirane SOC timove), i hardware zahtjevi (serveri s desecima GB RAM-a). Moj sustav cilja na "democratization" NSM-a - moguće pokrenuti na starom laptopu s minimalnim znanjem.

Četvrtak, 21. svibnja

Ukratko

- Pisanje poglavlja o implementaciji
- Dokumentiranje Docker konfiguracija

Danas sam pisao tehničke detalje implementacije. Prvo poglavlje pokriva Docker stack arhitekturu: kako su kontejneri međusobno povezani, koje volume mountove koriste, i kako network bridge omogućuje komunikaciju između njih.

Dokumentirao sam svaki Docker Compose servis posebno: Zeek (koje interface-e sluša, koje skripte su učitane, retention policy), Telegraf (input/output konfiguracija, parsing logike), InfluxDB (database setup, retention policies, user permissions), i Grafana (dashboard konfiguracija, data source povezivanje).

Pisanje ovog dijela me natjeralo da pregledam svoje trenutne konfiguracije. Uočio sam nekoliko nedosljednosti - neke environment varijable su hardkodirane umjesto da se koriste .env datoteke, neki volumei nisu optimalno mapirani. Napravio sam bilješke što moram ispraviti u kodu prije finalizacije.

Također sam pisao o CachyOS host konfiguraciji: hostapd.conf za AP setup, dnsmasq.conf za DHCP, i iptables rules za NAT/forwarding. Ovo je kritično jer korisnici koji žele replicirati sustav trebaju znati kako postaviti host prije pokretanja Docker stacka.

Shvatio sam da mi nedostaje dijagram arhitekture. Tekst objašnjava komponente, ali

vizualni prikaz bi bio mnogo jasniji. Moram napraviti dijagram koji pokazuje data flow: mrežni promet → host interface → Zeek kontejner → logovi → Telegraf → InfluxDB → Grafana dashboard.

Srijeda, 20. svibnja

Ukratko

- Pisanje poglavlja o pregledu postojećih rješenja
- Istraživanje komercijalnih NSM alata

Pisao sam poglavlje koje uspoređuje postojeća NSM rješenja s mojim pristupom. Morao sam istražiti što nude komercijalni alati (Splunk, QRadar, AlienVault) i open-source alternative (Security Onion, SELKS, pfSense).

Splunk je industry standard ali astronomski skup za male firme. Licensing model je baziran na volumenu podataka (GB per day) što ga čini neisplativim za kontinuirani monitoring. Također zahtijeva dedikirani server sa značajnim resursima.

Security Onion je najpopularniji open-source NSM stack. Dolazi s Zeekom, Suricata, Elasticsearch, i Kibana dashboardima. Problem je što je dizajniran za enterprise deployment - instalacija zahtijeva minimum 12GB RAM-a, kompleksna je za konfiguraciju, i targeting je SOC profesionalce, ne tehnički pismene laike.

Moj pristup se razlikuje po tome što je lightweight (radi na 4GB RAM-a), containerized (lako deployment i izolacija), i user-friendly (predkonfigurirani dashboardi, minimalna ručna konfiguracija). Trade-off je manji feature set - nemam IPS funkcionalnost, naprednu korelaciju, ili threat hunting alate koje Security Onion nudi.

Pisanje ovog dijela mi je pomoglo jasnije artikulirati value proposition mog rada: nije zamjena za enterprise SOC, već enabling tool za one koji inače ne bi imali NSM mogućnosti.

Utorak, 19. svibnja

Ukratko

- Pisanje uvodnog poglavlja
- Definiranje motivacije i ciljeva

Započeo sam pisanje uvodnog poglavlja koje objašnjava motivaciju za ovaj rad. Ključna točka: male firme i privatni korisnici nemaju pristup enterprise-level network security monitoring alatima zbog cijene i kompleksnosti, ali su jednako ranjivi na napade kao i velike korporacije.

Statistika koju sam pronašao: 43% cyber napada cilja mala preduzeća, ali samo 14% ima adekvatnu zaštitu. Glavni razlozi: nedostatak budžeta za security alate i nedostatak ekspertize za njihovu implementaciju. Moj sustav pokušava riješiti oba problema - besplatan je (open-source komponente) i jednostavan za deployment (Docker Compose + predkonfigurirani dashboardi).

Ciljevi rada koje sam definirao: (1) Implementirati funkcionalan NSM sustav koristeći Zeek i open-source alate, (2) Omogućiti deployment na commodity hardware (laptop, mali server), (3) Pružiti intuitivne dashboarde za monitoring mrežnog prometa, (4) Evaluirati

različite backend opcije (InfluxDB vs Elasticsearch), (5) Istražiti mogućnosti automatske reakcije na prijetnje.

Također sam pisao o scope limitation-ima - što ovaj rad ne pokriva. Ne bavim se host-based detekcijom (endpoint security), ne implementiram puni SOC workflow (incident response, forensics), i ne razvijam nove detekcijske algoritme (koristim postojeće Zeek mogućnosti).

Uvod je kritičan jer postavlja očekivanja i kontekst za ostatak rada. Želim izbjeći situaciju gdje čitatelj očekuje enterprise-level sustav, a dobiva proof-of-concept za mala okruženja.

Ponedjeljak, 18. svibnja

Ukratko

- Strukturiranje diplomskog rada
- Kreiranje outline-a i referenci

Počeo sam s definiranjem strukture diplomskog rada. Poglavlja: 1. Uvod, 2. Teorijske osnove (NSM, Zeek arhitektura, time-series baze), 3. Pregled postojećih rješenja, 4. Arhitektura sustava, 5. Implementacija, 6. Evaluacija, 7. Zaključak i budući rad.

Kreirao sam LaTeX template s potrebnim paketima (hyperref za linkove, listings za code snippets, graphicx za dijagrame). Postavio sam bibliografiju - trebam reference za Zeek paper, InfluxDB dokumentaciju, Docker best practices, i NSM metodologiju (Bejtlich knjige).

Također sam definirao strukture koje će se ponavljati kroz rad: code listings će imati konzistentan formatting (syntax highlighting, line numbers), dijagrami će koristiti isti stil (boje, fontove), i tablice će imati uniform layout.

Jedan problem koji sam uočio: nemam dovoljno screenshot-ova Grafana dashboarda. Trenutno imam samo basic grafove, ali za diplomski trebam pokazati finalne, polished dashboarde s realnim podacima. Moram generirati promet i napraviti kvalitetne screenshot-ove. Kreirao sam TODO listu za svako poglavlje - što mi još nedostaje u implementaciji da mogu pisati o tome. Npr. za poglavlje Evaluacija trebam benchmark rezultate, za Implementaciju trebam čiste Docker Compose datoteke, za Arhitekturu trebam dijagrame.

Nedjelja, 17. svibnja

Ukratko

- Istraživanje literature za diplomski
- Prikupljanje referenci

Proveo sam dan tražeći relevantnu literaturu za diplomski rad. Trebam academic papers o NSM-u, tehničku dokumentaciju Zeeka, i best practice guide-ove za Docker deploymente. Našao sam nekoliko ključnih resursa: Bejtlich-eva "The Practice of Network Security Monitoring" (biblija NSM-a), Zeek documentation (<https://docs.zeek.org>), InfluxDB vs Elasticsearch comparisons (nekoliko blog postova i technical reports), i Docker security best practices (od Docker Inc i NIST).

Problem s akademskim paperima je što su većina fokusirani na novel research (novi algoritmi za detekciju, machine learning pristupe), a ne na praktičnu implementaciju existing tools. Moj rad je više "systems engineering" nego research, pa moram biti oprezan kako to prezentiram.

Također sam tražio statistics o cyber security u malim firmama - trebam podatke koji podržavaju motivaciju rada. Pronašao sam nekoliko izvještaja od Verizon DBIR i Cisco koji pokazuju da male firme su sve češće meta napada, ali nemaju resurse za zaštitu.

Kreirao sam Zotero biblioteku za upravljanje referencama. Sve što pročitam i planiram citirati dodajem tamo s notama zašto je relevantno. To će ubrzati pisanje bibliografije kasnije.

Subota, 16. svibnja

Ukratko

- Planiranje poglavlja o automatskoj reakciji
- Istraživanje Python watchdog modula

Iako još nisam implementirao automatsku reakciju, moram planirati kako će to poglavlje izgledati u diplomskom. Teorijski, automatska reakcija ima nekoliko komponenti: detekcija (Zeek generira notice.log), odlučivanje (Python skripta evaluira je li to stvarna prijetnja), i akcija (blokiranje IP-a, slanje upozorenja).

Istraživao sam watchdog Python modul detaljnije. Dokumentacija pokazuje kako postaviti observer za file system events. Za moj use case, trebam observer na Zeek log direktoriju koji triggera callback kad se notice.log modificira. Callback čita novi redak, parsira ga, i odlučuje treba li reagirati.

Problem koji sam anticipirao: race conditions. Što ako Zeek piše u notice.log istovremeno kad Python skripta čita? Trebam mehanizam za file locking ili buffering. Jedna opcija je koristiti tail -F umjesto watchdog-a - tail automatski čeka da se pisanje završi prije čitanja.

Za diplomski, planiram napisati pseudo-code algoritma za automatsku reakciju, čak i ako implementacija nije kompletna. To će pokazati razumijevanje problema i planiran pristup rješavanju. Također ću dokumentirati design decisions (zašto log parsing umjesto Broker-a, zašto watchdog umjesto polling-a).

Cilj je da čitatelj može uzeti moj rad i implementirati feature sam, čak i ako ja to ne stignem do obrane.

Tjedan 11

(9. svibnja – 15. svibnja)

Petak, 15. svibnja

Ukratko

- Usporedba InfluxDB i Elasticsearch arhitektura
- Evaluacija storage modela

Nastavio sam istraživanje razlika između InfluxDB-a (koji trenutno koristim) i Elasticsearch-a (koji sam ranije testirao). Fokus je bio na arhitekturi podataka i kako svaki sustav optimizira storage.

InfluxDB je time-series baza što znači da je optimizirana za podatke koji dolaze sekvencijalno s timestamp-om. Svaki zapis ima time, tags (indeksirani metapodaci), i fields (vrijednosti). Za moj use case (Zeek conn.log koji se zapisuje kontinuirano), InfluxDB prirodno mapira strukturu: timestamp iz Zeeka postaje time, source/destination IP-ovi su tagovi, a bytes transferred je field.

Elasticsearch je document store koji tretira svaki log entry kao JSON dokument. Nema inherentnog koncepta vremena - timestamp je samo još jedno polje koje se može indeksirati. Prednost je fleksibilnost: mogu pohraniti potpuno različite strukture podataka u istu bazu (conn.log, dns.log, http.log) bez predefiniranja scheme.

Storage efficiency test: Uzeo sam 1GB conn.log podataka i usporedio veličinu nakon ingesta. InfluxDB: 450MB (kompresija 55%). Elasticsearch: 1.2GB (overhead zbog inverted index-a). Za moj laptop setup sa ograničenim prostorom, InfluxDB je jasna prednost.

Međutim, uočio sam da Elasticsearch ima bolju full-text search nad string poljima (npr. traženje specifičnih domena u dns.log), dok InfluxDB to ne podržava dobro jer su stringovi tretirani kao tagovi s ograničenom kardinalnosti.

Četvrtak, 14. svibnja

Ukratko

- Istraživanje query performansi
- Analiza aggregation mogućnosti
- Testiranje kompleksnih upita

Testirao sam query performanse oba sustava na istom datasetu. Cilj je vidjeti koji je brži za tipove upita koje moj dashboard zahtijeva (top N destination IP-ova, broj veza po minuti, anomalije u prometu).

InfluxDB koristi InfluxQL ili Flux query jezik. Za jednostavne agregacije (COUNT, SUM, MEAN) po vremenu, performanse su odlične - upiti se izvršavaju u <100ms čak i na velikim datasetima. Problem nastaje kad trebam kompleksne JOIN-ove između različitih measurement-a (npr. korelacija conn.log i dns.log). InfluxDB to ne podržava prirodno - moram raditi klijentsku agregaciju.

Elasticsearch koristi Query DSL (JSON-based). Aggregation framework je moćniji - mogu raditi nested aggregations, terms buckets, i kombinirati više izvora podataka u jednom upitu. Ali, latencija je veća - isti upiti traju 300-500ms. Za dashboard koji se refresha

svakih 30 sekundi, to nije problem, ali za real-time reakciju možda jest.

Zaključak do sada: InfluxDB je bolji za moj trenutni use case (time-series dashboarding, minimalni resursi). Elasticsearch bi bio bolji ako mi treba napredna korelacija logova i full-text search, ali to dolazi s cjenom većeg memorijskog otiska.

Za diplomski rad, planiram zadržati InfluxDB kao default, ali dokumentirati kako korisnici mogu switchati na Elasticsearch ako im trebaju naprednije mogućnosti.

Srijeda, 13. svibnja

Ukratko

- Istraživanje retention policy razlika
- Analiza data lifecycle management-a
- Evaluacija downsampling strategija

Jedan od ključnih aspekata za male firme i privatne korisnike je koliko dugo mogu zadržati podatke prije nego zauzmu previše prostora. Istraživao sam kako svaki sustav rukuje retention i downsampling-om.

InfluxDB ima ugrađene retention policies. Mogu definirati da conn.log drži sirove podatke 7 dana, zatim automatski agregira na 1-minutne prosjeke i drži to 30 dana, i na kraju briše sve starije od toga. To je idealno za moj scenarij gdje laptop nema puno diska - detaljni podaci za skorašnju analizu, agregirani za povijesni trend.

Elasticsearch ne dolazi s built-in retention mehanizmom. Moram koristiti Index Lifecycle Management (ILM) koji zahtijeva dodatnu konfiguraciju. ILM omogućuje rollover (kreiranje novog indexa nakon određene veličine/vremena), shrinking (smanjivanje broja shardova), i deletion. Kompleksnije je postaviti, ali i fleksibilnije.

Testirao sam downsampling na oba sustava. InfluxDB continuous queries automatski agregiraju podatke u pozadini - npr. svaki sat izračuna prosjek prometa i spremi u novi measurement. Elasticsearch zahtijeva Transform job koji također radi agregaciju, ali s više overhead-a.

Za use case malih firmi koje nemaju dedikirani storage i trebaju zadržati podatke dugoročno bez velikih troškova, InfluxDB retention + downsampling je jednostavniji za konfigurirati i održavati.

Utorak, 12. svibnja

Ukratko

- Istraživanje centraliziranog repozitorija za logove
- Evaluacija crowd-sourced threat intelligence pristupa
- Analiza opt-in/opt-out mehanizama

Mentor je predložio zanimljivu značajku: centralni repozitorij gdje korisnici mogu slati anonimizirane logove i dobivati heuristiku natrag. Ideja je smanjiti false positive rate kroz crowd-sourced inteligenciju - ako 1000 korisnika vidi isti pattern koji nije prijetnja,

sustav to nauči.

Istraživao sam postojeće implementacije ovog koncepta. VirusTotal radi nešto slično za file hash-eve - korisnici uploadaju sumnjiive fajlove, svi dobivaju pristup rezultatima skeniranja. Za mrežni promet, AlienVault OTX nudi community-driven threat intelligence feed.

Problem s logovima je privatnost. Zeek conn.log sadrži source i destination IP adrese koje mogu otkriti što korisnik radi. Prije slanja u centralni repozitorij, morao bih anonimizirati osjetljive podatke: zamjenu privatnih IP-ova s placeholder-ima, hashing domena (da zadržim pattern ali ne točan domain), i uklanjanje payload podataka.

Statistika koju bih skupljao: koliko puta se određeni pattern pojavio (npr. connection na port 8082), koliko korisnika je generiralo notice za taj pattern, i koliko ih je označilo kao false positive. Heuristika: ako 80% korisnika označi pattern kao false positive, sustav automatski ga dodaje na whitelist.

Opt-in/opt-out mehanizam mora biti jasan i transparentan. Default bi bio opt-out (privatnost first), s jasnim objašnjenjem što se šalje i kako se koristi ako korisnik odluči sudjelovati. GDPR compliance je kritičan ako planiram nuditi ovo u EU.

Ponedjeljak, 11. svibnja

Ukratko

- Prijepis sastanka
- Početak pisanja diplomskog rada

Danas je održan sastanak s mentorom zbog dogovora oko daljnjeg razvoja diplomskog rada. Pod područjima za istraživanje definirani su:

- istraživanje o razlikama, prednostima i manama korištenja InfluxDB i ElasticSearch baza podataka
- Definiran dodatna značajka eliminacije alarma (smanjivanje broja "false positiva")
 - Centralni repozitorij za logove + statistika i potom heuristika korištenja (uključujući i opt-in/out za skupljanje podataka)

Cilj je omogućiti i usmjeriti uslugu privatnim korisnicima i malim firmama što uključuje dijelove SOC aktivnosti i procesa te omogućavanje nadzora i kontrole tehnički pismenoj osobi. Započeto pisanje diplomskog rada.

Nedjelja, 10. svibnja

Ukratko

- Daljnja optimizacija repozitorija

Nastavak optimizacije od jučer. Morao sam se pobrinuti da su nepotrebni "dependency" paketi uklonjeni.

Subota, 9. svibnja

Ukratko

- Čišćenje repozitorija i alata

- - - - -

Išao sam sekvencijski micati alate iz repozitorija kako bi olakšao cijelokupni obujam direktorija. Također mobitel sa rootom je ostao u Rijeci pa ne mogu raditi t50 i hping3 sa mobitela više.

Maknuo sam .spicy skripte i parsiranje Radius protokola. Iako korisno u mojem slučaju ($ISP \rightarrow_{ETH} \rightarrow LokalniRuter \rightarrow_{BUS} \rightarrow AP \rightarrow_{WiFi} \rightarrow Ostali\ klijenti$) ne mogu očekivati da će se svi spajati preko Radius protokola.

Također i zeek2es skripte jer sam shvatio da ga limitiram ne koristeći Elasticsearch. Riješiti ću to drugačije kako ne bih dodavao nepotreban bloat na projekt.

Za sada se Zeek Broker + ZPM miču zbog redundancije.

Tjedan 10

(2. svibnja – 8. svibnja 2026.)

Petak, 8. svibnja

Ukratko

- Validacija postojećih Zeek konfiguracija
- Provjera Docker volume mountova
- Testiranje log rotacije

Dan posvećen validaciji postojećeg setup-a. S obzirom da nisam imao vremena za nove implementacije, fokusirao sam se na provjeru radi li sve kako treba.

Prvo sam validirao Docker volume mountove između kontejnera. Zeek kontejner zapisuje logove u shared volume koji Telegraf čita. Provjerio sam permisije i vlasništvo datoteka - sve je ispravno postavljeno. Telegraf uspješno čita nove log zapise kako Zeek stvara ih. Log rotacija je potencijalni problem koji sam uočio. Zeek po defaultu rotira logove svakih sat vremena, što znači da se stari conn.log preimenuje u conn.log.1, a novi conn.log se kreira. Telegraf mora biti svjestan ovog ponašanja inače propušta podatke. Provjerio sam Telegraf konfiguraciju - koristi tail input plugin s `from_beginning = true` opcijom što znači da prati datoteku čak i nakon rotacije.

Testirao sam i što se događa kad InfluxDB kontejner padne ili se restarta. Telegraf ima buffer mehanizam koji čuva podatke lokalno dok InfluxDB nije dostupan, pa onda šalje sve odjednom. To je dobro za kontinuitet podataka, ali može stvoriti spike u Grafana dashboardu ako padne na dulje vrijeme.

Nisu bile velike izmjene, ali validacija postojećeg sustava je bitna prije nego počnem dodavati nove komponente poput automatske reakcije.

Četvrtak, 7. svibnja

Ukratko

- Čitanje dokumentacije o Zeek skriptama
- Analiza notice framework-a
- Planiranje detekcijskih pravila

Proveo sam dan u Zeek dokumentaciji pokušavajući bolje razumjeti kako notice framework funkcionira. To je ključno za implementaciju automatske reakcije jer notice.log je centralno mjesto gdje Zeek šalje sva upozorenja.

Notice framework omogućuje definiranje custom upozorenja kroz Zeek skripte. Svako upozorenje ima type (npr. `Scan::Port_Scan`), message (opis što se dogodilo), i dodatne metadata poput source IP-a i vremena. Problem je što default Zeek instalacija dolazi s minimalnim brojem detekcijskih skripti - većina prijetnji neće generirati notice bez dodatnih skripti.

Proučavao sam kako napisati pravilo za detekciju SYN flood napada. Logika je jednostavna: prati broj novih SYN paketa po source IP-u u određenom vremenskom prozoru, i ako premaši threshold, generiraj notice. Zeek koristi SumStats framework za agregaciju podataka kroz vrijeme, što je idealno za ovakve scenarije.

Još jedan zanimljiv koncept iz dokumentacije je Intelligence framework. Omogućuje

učitavanje liste poznatih malicioznih IP adresa, domena ili hasheva, i automatsko generiranje notice-a kad Zeek vidi match. To bi bilo korisno za integraciju s threat intelligence feedovima koje sam analizirao ranije (The DFIR Report).

Zaključak: trebam napisati nekoliko custom Zeek skripti koje će generirati notice-e za najčešće napade (port scanning, SYN flood, suspicious DNS queries). To će biti input za Python skriptu koja izvršava automatsku reakciju.

Srijeda, 6. svibnja

Ukratko

- Proučavanje Zeek Broker dokumentacije
- Evaluacija real-time komunikacije
- Usporedba Broker vs. log parsing pristupa

Istraživao sam Zeek Broker framework kao alternativu čitanju log datoteka za automatsku reakciju. Broker je Zeek-ov messaging sistem koji omogućuje real-time komunikaciju između Zeek procesa i vanjskih aplikacija.

Prednost Broker pristupa je latencija - umjesto da Python skripta čeka da se log zapiše na disk, parsira ga i reagira, Broker omogućuje slanje eventa direktno iz Zeeka u Python aplikaciju čim se dogodi. To znači da reakcija može biti gotovo trenutna.

Tehnički, trebao bih napisati Zeek skriptu koja šalje Broker event kad detektira prijetnju, i Python listener koji prima te eventove koristeći broker Python binding. Problem je što Broker zahtijeva kompleksniju setup - trebam otvoriti dodatni port na Zeek kontejneru za komunikaciju, i osigurati da Python aplikacija može pristupiti tom portu.

Usporedio sam oba pristupa za moj use case: - Log parsing (zeek2es): Jednostavniji, ne zahtijeva promjene u Zeek kontejneru, ali latencija 1-5 sekundi. - Broker: Brži (latencija <1 sekunda), ali kompleksnija implementacija i debugging.

Za proof-of-concept, log parsing pristup je dovoljan. Broker mogu dodati kasnije ako se ispostavi da je latencija problem. Realno, za većinu prijetnji koje planiram detektirati (SYN flood, port scanning), par sekundi kašnjenja u reakciji nije kritično.

Utorak, 5. svibnja

Ukratko

- Analiza postojećih Zeek skripti iz package managera
- Istraživanje ZPM (Zeek Package Manager, zkg)
- Evaluacija community skripti

Umjesto da pišem sve Zeek skripte od nule, istraživao sam što već postoji u Zeek community. ZKG (Zeek Package Manager) je centralni repozitorij za Zeek skripte i pluginove.

Našao sam nekoliko korisnih paketa: ja3 (TLS fingerprinting koji sam spominjao ranije za detekciju uređaja bez obzira na MAC), hassh (SSH fingerprinting), i cve-2020-0601 (detekcija specifičnog Windows exploit-a). Instalacija je jednostavna kroz zkg install komandu unutar Zeek kontejnera.

Problem je što mnogi paketi nisu održavani ili su dizajnirani za starije verzije Zeeka. Testirao sam nekoliko njih - neki rade bez problema, neki bacaju errore pri učitavanju. Moram biti oprezan s dependency-jima jer dodavanje previše skripti može usporiti Zeek processing.

ja3 paket je posebno zanimljiv za moj problem MAC randomizacije. Instalirao sam ga i testirao na prometu preko AP-a. Zeek sada generira ssl.log s dodatnim poljem ja3 koje sadrži hash TLS client hello-a. To znači da mogu identificirati uređaj čak i kad mijenja MAC adresu - isti uređaj će uvijek imati isti ja3 hash.

Za implementaciju automatske reakcije, planiram koristiti kombinaciju default Zeek detekcija + nekoliko community paketa (ja3, hassh) + custom skripte koje ću napisati za specifične use case-ove mog sustava.

Ponedjeljak, 4. svibnja

Ukratko

- Istraživanje iptables integracije s Python skriptama
- Proučavanje subprocess modula
- Planiranje arhitekture za blocking mehanizam

- - - - -

Fokusirao sam se na drugu polovicu automatske reakcije - izvršavanje akcija nakon detekcije prijetnje. Najjednostavnija akcija je blokiranje IP adrese kroz iptables na CachyOS host-u.

Python subprocess modul omogućuje pozivanje shell komandi direktno iz Python skripte. To znači da mogu napisati funkciju koja prima IP adresu kao parametar i izvršava iptables -A INPUT -s `!IP` -j DROP. Problem je što iptables komande zahtijevaju root privilegije. Istraživao sam nekoliko pristupa kako to riješiti: - Pokretanje Python skripte kao root (loša security praksa) - Korištenje sudo s NOPASSWD za specifične iptables komande (bolji pristup, ali zahtijeva modifikaciju sudoers) - Pisanje privilegiranog servisa (systemd unit) koji prima komande preko socket-a (najsigurniji ali najkompleksniji)

Za proof-of-concept, koristit ću sudo pristup. U produkcijskom okruženju bih otišao sa systemd servisom.

Još jedan detalj koji moram riješiti je "unblocking". Ako automatski blokiram IP, trebam i mehanizam da ga odblokujem nakon određenog vremena ili ručno. To znači da Python skripta mora voditi evidenciju blokiranih IP adresa i timestampova, i periodički provjeravati treba li ih odblokirati.

Planiram koristiti SQLite bazu za ovu evidenciju - lightweight, ne zahtijeva dodatni Docker kontejner, i dovoljno brza za ovaj use case.

Nedjelja, 3. svibnja

Ukratko

- Istraživanje real-time log parsing biblioteka
- Evaluacija watchdog Python modula
- Testiranje file monitoring pristupa

- - - - -

Proučavao sam kako implementirati real-time čitanje Zeek logova bez konstantnog pollinga datoteka. watchdog Python biblioteka omogućuje monitoring datotečnog sustava i triggering event-a kad se datoteka modificira.

Ideja je postaviti watchdog observer na Zeek log direktorij (koji je mountan kao Docker volume). Kad Zeek zapiše novi redak u notice.log, watchdog detektira promjenu i triggera callback funkciju u mojoj Python skripti. Ta funkcija parsira novi redak (koristeći zeek2es ili direktno čitanje TSV formata) i odlučuje treba li reagirati.

Testirao sam watchdog na testnoj datoteci - radi kako treba. Problem je što watchdog triggerira event za svaku promjenu, što znači da će se callback pozvati čak i kad Zeek samo flushuje buffer bez dodavanja novih redaka. Moram dodati logiku koja provjeri je li se stvarno dodao novi log entry.

Alternativni pristup je korištenje tail -F komande kroz subprocess i čitanje stdout-a. To je jednostavnije, ali manje "pythonic" rješenje. Za proof-of-concept, tail pristup je vjerojatno brži za implementirati.

Zaključak: za prvu verziju automatske reakcije, koristit ću tail -F notice.log — python script.py pristup. watchdog mogu dodati kasnije ako tail pristup ne bude dovoljno robustan.

Subota, 2. svibnja

Ukratko

- Revizija dokumentacije zeek2es alata
- Testiranje JSON parsinga Zeek logova
- Evaluacija error handling strategija

- - - - -

Nastavio sam istraživanje zeek2es alata iz prošlog tjedna. Fokus danas bio je na praktičnim detaljima kako integrirati ga u Python skriptu za automatsku reakciju.

zeek2es pretvara Zeek TSV logove u JSON format koji je jednostavan za parsiranje. Testirao sam ga na različitim log tipovima (conn.log, dns.log, notice.log) - sve radi bez problema. JSON struktura je intuitivna: svako polje iz Zeek loga postaje ključ u JSON objektu.

Problem koji sam uočio je handling malformiranih log zapisa. Ako Zeek zapisuje podatke dok se paket još procesira, može se dogoditi da redak u logu bude nepotpun ili oštećen. zeek2es u tom slučaju baca exception koji bi srušio moju Python skriptu ako ga ne uhvatim.

Istraživao sam best practice za error handling u Python skriptama koje rade 24/7. Ključ je koristiti try-except blokove oko parsing logike, i logirati svaki error u posebnu datoteku za kasniju analizu. Ako jedan log redak ne može biti parsiran, skripta ga preskače i nastavlja s sljedećim.

Još jedna stvar koju sam primjetio u dokumentaciji je da zeek2es podržava direktno slanje u Elasticsearch ako je konfiguriran. To bi moglo biti korisno za arhiviranje svih Zeek logova u pretraživu bazu, ali za moj trenutni use case (real-time reakcija) nije potrebno. Plan je napisati jednostavnu Python skriptu koja čita notice.log kroz zeek2es, parsira JSON, i izvršava akciju (npr. print za testiranje) kad vidi specifičan tip prijetnje.

Tjedan 9

(25. travnja – 1. svibnja 2026.)

Petak, 1. svibnja

Ukratko

- Praznik rada

Praznik rada, danas nije ništa napravljeno.

Četvrtak, 30. travnja

Ukratko

- Istraživanje Python biblioteka za automatsku reakciju
- Analiza ZAT (Zeek Analysis Tools)
- Evaluacija zeek2es za JSON processing

Fokus dana bio je na istraživanju kako implementirati poluautomatsku reakciju na Zeek upozorenja koju mentor traži. Proučavao sam dva pristupa: ZAT biblioteku i zeek2es alat.

ZAT (Zeek Analysis Tools) izgleda kao najdirektnije rješenje. Biblioteka omogućuje čitanje Zeek logova direktno iz Pythona i rad s njima kao s pandas DataFrames. Primjeri s njihove dokumentacije pokazuju kako se može napraviti real-time monitoring - skripta čita conn.log ili notice.log i okida akcije na temelju definiranih pravila. Problem je što većina primjera pretpostavlja statičke log datoteke, a meni treba stream processing jer Zeek radi kontinuirano u Docker kontejneru.

zeek2es pristup je zanimljiviji za moj use case. Iako je primarno namijenjen za Elasticsearch integraciju, alat pretvara Zeek logove u JSON format koji se može procesirati bilo kojim Python skriptom. To znači da mogu napisati listener koji prati notice.log, parsira JSON output i šalje upozorenja ili izvršava akcije (npr. blokiranje IP adrese kroz iptables na CacherOS host-u).

Glavni izazov je kako spojiti ovo s postojećim stackom (Zeek → Telegraf → InfluxDB → Grafana). Jedna opcija je da Python skripta čita iste log datoteke koje Telegraf već procesira, ali to može dovesti do race condition problema. Druga opcija je koristiti Zeek's Broker framework za komunikaciju, ali to zahtijeva dublju integraciju.

Zaključak dana: zeek2es + Python listener izgleda kao najjednostavniji proof-of-concept za automatsku reakciju. Sljedeći korak je testirati ga na stvarnim Zeek logovima iz mog sustava.

Srijeda, 29. travnja

Ukratko

- Instalacija fuzzing alata (hping3, t50)
- Testiranje generatora prometa
- Evaluacija Termux okruženja za testiranje

Kako bi mogao testirati detekciju prijetnji i automatsku reakciju, trebam način da generiram kontrolirani "zlonamjerni" promet. Istraživao sam fuzzing alate koji mogu simulirati različite vrste napada preko mog AP-a.

hping3 je prvi alat koji sam instalirao. To je packet crafter koji omogućuje slanje custom TCP/UDP/ICMP paketa. Koristim ga za simulaciju SYN flood napada i testiranje hoće li Zeek detektirati anomaliju u conn.log. Prednost je što mogu precizno kontrolirati sve parametre paketa (source IP, flags, payload).

t50 je "multi-protocol injector" - brutalniji alat koji može generirati ogroman volumen prometa preko 15 različitih protokola istovremeno. Testirao sam ga u kontroliranom okruženju da vidim kako se moj Docker stack ponaša pod opterećenjem. Rezultat: Telegraf i InfluxDB nemaju problema, ali Zeek počinje gubiti pakete (capture_loss.log pokazuje 2% gubitka) kad t50 pošalje više od 50k paketa po sekundi.

Termux na mobitelu je zanimljiva opcija za mobilno testiranje. Instalirao sam ga i podigao osnovne networking alate. Ideja je koristiti mobitel kao "napadački" uređaj koji se spaja na moj AP, što bi simuliralo stvaran scenarij kompromitiranog uređaja u mreži. Ograničenje je što neki alati (poput t50) zahtijevaju root pristup koji je problematičan na Android-u.

Ovo je temelj za testiranje automatske reakcije - trebam kontrolirane prijetnje da mogu verificirati rade li Python skripte koje planiram napisati.

Utorak, 28. travnja

Ukratko

- Praktično testiranje fuzzing alata
- Analiza Zeek outputa pod napadom
- Dokumentiranje baseline ponašanja

Nastavio sam s testiranjem iz jučer. Pokrenuo sam hping3 sa svog laptopa prema vanjskom serveru (da vidim kako Zeek bilježi odlazni promet) i sa mobitela (preko Termux-a) prema laptopu (dolazni promet preko AP-a).

Zeek je uspješno detektirao SYN flood. U conn.log se pojavljuju stotine kratkih veza s istim source IP-om ali različitim portovima. notice.log nije generirao automatsko upozorenje jer nemam učitano odgovarajuću detekcijsku skriptu - to je nešto što moram dodati. Ovo potvrđuje da sam Zeek vidi promet ispravno, ali mu nedostaje inteligencija za prepoznavanje specifičnih napada.

Usporedio sam "normalan" conn.log (par sati obične browsing aktivnosti preko AP-a) s conn.log tijekom hping3 testa. Razlika je očita: normalni promet ima dulje veze (duration > 1s), veće količine prenesenih podataka, i raznolike destination portove. Napad ima kratke veze (duration ~ 0.001s), minimalne podatke, i sve ide na isti port.

Ovo mi je dalo ideju za prvu Python skriptu: pravilo koje gleda za anomaliju "više od 100 novih veza sa istog IP-a u manje od 10 sekundi prema istom portu". To je jednostavno pravilo koje pokriva SYN flood, a lako se implementira čitanjem conn.log kroz zeek2es. Dokumentirao sam baseline podatke (prosječan broj veza po minuti, top 10 destination portova, prosječno trajanje veze) jer to će biti referentna točka za detekciju anomalija.

Ponedjeljak, 27. travnja

Ukratko

- Postavljanje Termux okruženja
- Instalacija networking alata na mobitelu
- Prvi testovi s Android uređajem kao klijentom

- - - - -

Počeo sam s postavkom Termux-a na mobitelu. Ideja je imati mobilni "laboratorij" koji se može spojiti na moj AP i generirati kontrolirani promet bez potrebe za dodatnim hardverom. Instalacija je bila jednostavna - Termux iz F-Droid repozitorija, zatim pkg install za osnovne alate (nmap, curl, netcat).

hping3 instalacija je bila problematična jer zahtijeva kompajliranje iz izvora, a neki headeri nedostaju u Termux okruženju. Nakon traženja po forumima, našao sam da postoji precompiled verzija u Termux community repo, ali i dalje zahtijeva root za raw socket pristup. Odlučio sam koristiti Android rooting samo za testne svrhe, ne kao trajno rješenje. Prvi testovi bili su jednostavni - spojio mobitel na moj AP (koji vrti hostapd na CACHYOS-u) i provjerio vidi li ga Zeek. U conn.log se pojavljuje s MAC adresom mobitela i IP adresom iz dnsmasq DHCP raspona. dns.log bilježi sve upite koje mobitel šalje (puno Google servisa i Android connectivity checkova).

Ovo potvrđuje da moj setup ispravno hvata sve što se događa preko AP-a. Sljedeći korak je koristiti mobitel za simulaciju zlonamjernog ponašanja (port scanning, DoS pokušaji) i testirati hoće li Zeek upozorenja biti dovoljno precizna za automatsku reakciju.

Nedjelja, 26. travnja

Ukratko

- Analiza prometa iz prethodnog tjedan
- Identifikacija pattern-a za detekciju
- Planiranje arhitekture za automatsku reakciju

- - - - -

Proveo sam dan analizirajući Zeek logove koje sam skupljao kroz prethodni tjedan. Cilj je bio pronaći pattern-e koji bi mogli biti temelj za automatsku detekciju prijetnji.

U conn.log sam primijetio nekoliko zanimljivih anomalija: periodične kratke veze prema istim IP adresama u Rusiji (svaki put isti destination port 443), što izgleda kao beacon ponašanje. dns.log pokazuje da neki uređaji na AP-u konstantno resolvaju iste domene koje su sumnjive (kratki nazivi, TLD-ovi poput .top i .xyz). http.log bilježi User-Agent stringove koji ne odgovaraju OS-u koji bi uređaj trebao imati prema DHCP hostname-u. Ovo su sve signali koje mogu pretvoriti u detekcijska pravila. Problem je što trenutno nemam način da razlikujem legitiman od malicioznog ponašanja - trebam veći dataset "normalnog" prometa da mogu postaviti baseline.

Arhitektura za automatsku reakciju bi trebala izgledati ovako: Zeek generira logove → Python skripta (koristi zeek2es) čita notice.log u real-time → ako se detektira prijetnja, skripta šalje komandu na CACHYOS host (iptables DROP za blokiradu IP-a ili killall za

prekid veze) → istovremeno šalje notification u Grafana dashboard preko InfluxDB custom metrike.

Sljedeći tjedan moram napraviti proof-of-concept s jednostavnim pravilom (npr. blokiranje IP-a ako generiš više od 1000 veza u minuti) i testirati ga s fuzzing alatima.

Subota, 25. travnja

Ukratko

- Spajanje Zeeka na mrežu nakon izmjena
- Verifikacija Docker mreže
- Testiranje vidljivosti prometa kroz AP

- - - - -

Nakon prošlotjednih promjena u konfiguraciji, morao sam verificirati da Zeek kontejner ispravno vidi sav promet koji prolazi kroz laptop. Problem s Docker mrežom je što kontejneri po defaultu vide samo promet unutar Docker bridge mreže, ne i promet koji prolazi kroz host-ov network interface.

Rješenje je bilo prebaciti Zeek kontejner na host network mode (`-network host`). To znači da kontejner dijeli mrežni namespace s CachyOS host-om i vidi sve isto što vidi i host. Testirao sam s `tcpdump`-om na host-u i usporedio output s Zeek `conn.log` - svi paketi su prisutni.

Drugi test bio je spojiti mobitel na AP (`hostapd`) i generirati promet (streaming video, browsing). Zeek je uspješno bilježio sve HTTP i HTTPS veze. `dns.log` je pokazivao sve DNS upite koje mobitel šalje prije spajanja na servere. `http.log` je bilježio Host headere i User-Agent stringove.

Jedini problem bio je što Telegraf nije mogao čitati Zeek logove jer su vlasništvo Docker container usera. Riješio sam to dodavanjem Telegraf usera u odgovarajuću grupu i monitoranjem Zeek log direktorija kao shared volume između Zeek i Telegraf kontejnera.

Na kraju dana, cijeli stack (Zeek → Telegraf → InfluxDB → Grafana) radi kako treba. Grafana dashboardi pokazuju real-time podatke prometa koji prolazi kroz moj AP i ethernet vezu prema ISP-u.

Tjedan 8

(18. travnja – 24. travnja 2026.)

Petak, 24. travnja

Ukratko

- Implementacija Spicy parsera za Radius protokol
- Testiranje s Spicy Driverom

- - - - -

Nakon što sam shvatio kako Zeek "razgovara" s protokolima kroz Spicy framework, danas je bio dan za praktičnu implementaciju. Fokus je bio na Radius protokolu jer je relativno jednostavan za početak, a pritom rješava konkretan problem - autentifikaciju korisnika neovisno o MAC adresi u mom AP okruženju.

Proces je bio iterativan: pisanje .spicy datoteke s gramatikom protokola, zatim povezivanje s Zeekovim eventima kroz .evt datoteku. Ključni trenutak je bilo shvaćanje da svaki parser mora imati requires klauzulu za provjeru duljine paketa - bez toga dolazi do overflowa i parser pada na prvom pogrešno formatiranom paketu.

Spicy Driver se pokazao kao spasitelj živaca. Umjesto da svaki put pokrećem cijeli Zeek kontejner i čekam da procesira pcap, jednostavno pokrenem driver nad testnom datotekom i vidim gdje parser puca. To ubrzava razvoj s sat vremena po iteraciji na par minuta. Dan završavam s funkcionalnim parserom koji uspješno izvlači usernames iz Radius paketa. Plan je integrirati ga u svoj Zeek kontejner kako bi radius.log omogućio korelaciju korisnika preko DHCP-a, neovisno o tome koliko puta uređaji mijenjaju MAC adrese na mom AP-u. To će biti kritično za poluautomatsku detekciju prijetnji.

Četvrtak, 23. travnja

Ukratko

- Eksperimentiranje s ELK stackom
- Testiranje zeek2es alata
- Evaluacija potrebe za Elasticsearch-om u mom sustavu

- - - - -

Pokušao sam pokrenuti ELK stack na svom Acer Aspire 3 i odmah sam naišao na problem memorije. Sa 12GB RAM-a, Elasticsearch i Kibana zajedno zauzimaju oko 4-5GB u idle stanju. S obzirom da laptop funkcionira kao gateway (CachyOS host + Docker stack: Zeek, Telegraf, InfluxDB, Grafana), plus hostapd i dnsmasq za AP, to je previše.

Testirao sam zeek2es koji pretvara Zeek logove u JSON format. Shvatio sam da za moje trenutne potrebe - dashboarding preko Grafane - JQ filtriranje JSON-a daje istu funkcionalnost bez memorijskog opterećenja. Telegraf ionako već čita Zeek logove i šalje ih u InfluxDB, tako da dodavanje Elasticsearch-a u stack bi bilo redundantno za osnovni monitoring.

Ipak, zadržavam ELK kao opciju za naprednije korelacije kad počnem implementirati poluautomatsku reakciju na prijetnje. Elasticsearch bi mogao poslužiti za kompleksne upite koji kombiniraju više Zeek logova istovremeno (npr. korelacija DNS, HTTP i conn.log za detekciju C2 komunikacije).

Najvažnije otkriće dana: UID (jedinstveni identifikator sesije) omogućuje praćenje cijele priče jednog uređaja kroz različite Zeek logove, bez obzira na MAC randomizaciju. To mogu iskoristiti u Grafana dashboardu za prikaz "životnog ciklusa" sumnjive veze.

Srijeda, 22. travnja

Ukratko

- Analiza CTI izvještaja (The DFIR Report)
- Pretvaranje TTPs u Zeek upite
- Postavljanje izoliranog analitičkog okruženja

- - - - -

Današnji fokus je bio na praktičnoj primjeni threat intelligence izvještaja. Uzeo sam jedan od izvještaja s The DFIR Report koji opisuje Trickbot infekciju i pokušao ga prevesti u konkretne Zeek upite. TTPs iz izvještaja mogu se pretvoriti u Zeek skripte koje automatski generiraju notice.log eventi.

Umjesto da tražim specifične IP adrese (koje se mijenjaju), fokusirao sam se na ponašanje. Izvještaj je govorio o RDP-u preko nestandardnih portova, pa sam napravio upit koji traži sve conn.log zapise gdje je service=="rdp" ali resp_p != 3389. Sljedeći korak je napisati Zeek skriptu koja će takve anomalije automatski detektirati i slati upozorenje u moj Grafana dashboard preko InfluxDB metrike.

Još jedan detalj iz izvještaja bio je korištenje "staged" domena za C2 komunikaciju. Zeek dns.log omogućuje povijesnu pretragu - mogu vidjeti je li netko preko mog AP-a ikad kontaktirao te domene, čak i ako su sada ugašene.

Druga polovica dana bila je posvećena sigurnosti analitičkog okruženja. Budući da radim na CachyOS-u, pokušao sam replicirati Windows Sandbox pristup kroz Firejail. Sumnjive pcap datoteke otvaraju se u izoliranom okruženju jer Wireshark i Brim imaju historie ranjivosti. Firejail se pokazao kao dobar kompromis - brz je za pokretanje, izolira mrežu i datotečni sustav, a ne konzumira resurse kao puna VM. To je važno jer mi laptop već radi kao gateway s Docker stackom.

Utorak, 21. travnja

Ukratko

- Video 10: Taksonomija NSM podataka
- Video 11: Spicy Driver debugging
- Video 12: Log management s zeek2es

- - - - -

Dan je započeo s razumijevanjem Bejtlicheve taksonomije podataka. Uvijek mi je bilo nejasno zašto sustavi drže i pcap i logove kad logovi sadrže sve bitno. Danas sam shvatio - svaki tip podataka služi različitoj svrsi u mom sustavu. PCAP bi bio "nulti dokaz" za detaljnu analizu, Zeek logovi (koje Telegraf čita) su za brzu korelaciju i dashboarding u Grafani, ekstrahirane datoteke su za potencijalni sandbox, a IDS upozorenja (notice.log) su polazna točka za automatsku reakciju.

CloudShark primjer iz videa pokazuje kako idealno sučelje treba izgledati - klikneš na upozorenje i odmah vidiš pripadajuće pakete. To pokušavam postići s mojim stackom, ali trenutno imam samo Grafana dashboard koji prikazuje agregirane podatke iz InfluxDB-a. Trebam dodati vezu na PCAP datoteke za dubinsku analizu.

Video 11 me naučio pravilnom pristupu razvoju Spicy parsera. Umjesto da pišem kod naslijepo u Zeek kontejneru, Spicy Driver omogućuje brzo testiranje izvan Docker okruženja. Najbitniji dio je bio razumijevanje requires klauzule - svaki parser mora provjeriti duljinu paketa prije parsiranja.

Log management video me natjerao na razmišljanje o skalabilnosti. Trenutno imam relativno male količine podataka (promet preko mog AP-a i laptop prometa), ali jednom kad počnem producijski monitoring veće mreže, logovi će rasti eksponencijalno. Moj trenutni stack (Zeek → Telegraf → InfluxDB → Grafana) je dovoljan, ali zeek2es + JQ kombinacija može biti korisna za ad-hoc analize van dashboarda.

Ponedjeljak, 20. travnja

Ukratko

- Video 6: Monitoring bežičnih mreža
- Video 7: Statistika gubitka paketa
- Video 8: Instalacija Zeeka
- Video 9: Uvod u Spicy framework

- - - - -

Prvi radni dan novog ciklusa učenja. Videi 6-9 su bili tehnički intenzivni, posebno dio o bežičnim mrežama. Shvatio sam da pokušaj sniffanja WiFi prometa direktno iz zraka nije rješenje - previše komplikacija s kanalima i enkripcijom. Moj pristup (postavljanje Zeek senzora na CachyOS host-u gdje se agregira promet s AP-a i ethernet veze prema ISP-u) je ispravna arhitektura. Zeek vidi sve što prođe kroz laptop koji funkcionira kao gateway.

Najvažniji dio dana bio je razumijevanje capture_loss mehanizma. Zeek prati TCP sekvencijske brojeve i detektira izgubljene pakete. To je kritično za moj sustav - ako gubim pakete između izvora i Zeek kontejnera, moji Grafana dashboardi pokazuju nepotpune podatke, a ekstrahirane datoteke su korumpirane. Za UDP i ICMP nema takve zaštite, što znači da moram paziti na konfiguraciju Docker mreže i bridge-a.

Instalacija Zeeka je potvrdila da je Docker pristup ispravan - koristim službene image-e umjesto kompajliranja. Najvažniji korak je bio testiranje - usporedio sam Zeek output s tcpdump zahvatom na host sustavu da potvrdim da kontejner ispravno vidi promet koji prolazi kroz laptop.

Uvod u Spicy mi je otvorio mogućnosti proširenja. Do sada sam mislio da sam ograničen na protokole koje Zeek već razumije, ali Spicy omogućuje pisanje potpuno novih parsera. To će biti ključno ako naletim na custom protokole u prometu koji prolazi kroz moj AP.

Nedjelja, 19. travnja

Ukratko

- Video 1: Temeljna filozofija NSM-a
- Video 2: Analiza kompromitacije (Mike's PC)
- Video 3: Trickbot case study

- Video 4: Postavljanje senzora
- Video 5: Pasivna detekcija asseta

Kroz nekoliko sljedećih dana (pretpostavljam danas i sutra) ću se pozabaviti ovom listom na youtuubu koju je postavio kanal Zeek (pretpostavljam netko od razvojnih programera Zeeka).

Lista: Zeek in Action by Zeek

Prvi dan analize videa 1-5 bio je ključan za razumijevanje metodologije. Bejtlich je brutalno jasan - NSM nije igranje s alatima već sustavno prikupljanje dokaza. To me natjeralo da preispitam svoj pristup - do sada sam fokusiran na dashboarding (Grafana grafovi: trenutne veze, najpopularnije odredište/izvorište, notice.log), ali nisam razmišljao o forenzičkoj vrijednosti i povezivanju podataka za analizu incidenata.

Najvažnije otkriće: hijerarhija izvora podataka. U mom sustavu to znači: krenem od Grafana dashboarda koji agregira conn.log podatke iz InfluxDB-a da vidim anomalije, zatim se spustim na Zeek HTTP i DNS logove za detalje, a tek na kraju koristim Wireshark za PCAP analizu. Prije sam radio obrnuto - skakao direktno u Wireshark i gubio vrijeme.

Problem MAC randomizacije koji me muči od početka projekta napokon ima rješenje - korelacija kroz DHCP hostname (dnsmasq na mom AP-u) i JA3 fingerprinting. Uređaj može mijenjati MAC koliko hoće, ali njegov TLS handshake ostaje konzistentan. To moram implementirati u Zeek skripte kako bi Grafana dashboard prikazivao stvarne uređaje, ne nasumične MAC adrese.

Trickbot case study mi je pokazao koliko je važno poznavati "normalno" ponašanje. Anomalija (nestandardni port 8082) je očita samo ako znam baseline. To znači da trebam provesti više vremena u pasivnom promatranju prometa preko mog AP-a i izgraditi profil normalnog ponašanja prije nego što mogu efikasno detektirati prijetnje.

Subota, 18. travnja

Ukratko

- Smanjivanje memorijskog otiska projekta

Daljnijim istraživanjem uvidjelo se da i InfluxDB nudi izgradnju "kontrolne ploče". Sučelje je slično i nudi slične opcije no neke vrijednosti se ne mogu odabrati kao "nositelji stupaca" najčešće stringovi poput IP adresa. Uz to InfluxDB nudi manje grafova sveukupno što znači da nije moguće na razumljiv i jasan način prikazati podatke kao što je to slučaj u Grafani. Iako je prvobitan plan bio dodavanje novih funkcionalnosti Zeeku također je pritom bio i pokušaj optimizacije cijelog projekta. Trenutno to je jedino moguće traženjem novog prihvatljivog rješenja (nešto što Grafana može čitati a taj "nepoznati" alat može čitati json datoteke).

Zaključak dana je da se trenutni "stack" neće optimizirati do kraja projekta.

Tjedan 7

(11. travnja – 17. travnja 2026.)

Petak, 17. travnja

Ukratko

- Finalizacija osnovnog Grafana dashboarda
- Testiranje s realnim mrežnim prometom
- Dokumentacija Flux upita i panela

Dashboard je konačno funkcionalan i prikazuje osnovne metrike iz `zeek_conn` i `zeek_notice` measurementa. Kreirao sam četiri osnovna panela: (1) vremenski graf broja konekcija po minuti, (2) tablicu top 10 destinacijskih IP adresa, (3) distribuciju protokola (TCP/UDP/ICMP), i (4) alert panel za Zeek notice zapise. Svaki panel koristi Flux upite s `range(start: -1h)` za prikaz zadnjeg sata podataka. Testirao sam s normalnim prometom (browsing, YouTube) i generiranim prometom (nmap scan, SSH brute-force simulacija). Svi eventi su se pravilno prikazali u panelima s latencijom od 5-10 sekundi. Dashboard sam exportao u JSON i spremio u `grafana/provisioning/dashboards/` kako bi se automatski učitavao pri pokretanju. Milestone M2 je sada potpuno verificiran s vizualizacijom podataka.

Četvrtak, 16. travnja

Ukratko

- Kreiranje prvog Grafana panela za konekcije
- Debugging Flux upita i vizualizacija problema
- Dodavanje filtera za protokole

Počeo sam s najjednostavnijim panelom - time series grafom ukupnog broja konekcija. Početni upit je bio problematičan jer nisam koristio `aggregateWindow()` funkciju pa su se svi eventi prikazivali kao pojedinačne točke bez agregacije. Nakon istraživanja Grafana-InfluxDB integracije, shvatio sam da moram grupirati podatke u vremenske intervale. Konačni upit koristi `aggregateWindow(every: 1m, fn: count)` za brojanje konekcija po minuti. Dodao sam i `filter()` za odvajanje TCP, UDP i ICMP prometa korištenjem `r.proto` polja iz `conn.log` zapisa. Panel prikazuje tri linije (po jedna za svaki protokol) s različitim bojama. Problem je bio što sam inicijalno koristio `r._value` umjesto `r.proto`, što je vraćalo greške u parsiranju.

Srijeda, 15. travnja

Ukratko

- Eksperimentiranje s Flux upitima u InfluxDB Data Explorer
- Identifikacija relevantnih polja iz `zeek_conn` measurementa
- Testiranje agregacijskih funkcija

Prije nego što sam počeo raditi u Grafani, htio sam se upoznati s InfluxDB Data Explorer sučeljem i testirati Flux upite direktno. Koristio sam

```
from(bucket: "zeek_logs") |> range(start: -1h) |> filter(fn:
  (r) => r._measurement == "zeek_conn")
```

kao bazni upit i eksperimentirao s različitim filterima. Otkrio sam da Telegraf sprema sva Zeek polja kao tagove ili fieldove - važna polja su `id_orig_h` (source IP), `id_resp_h` (destination IP), `id_resp_p` (destination port), `proto`, i `conn_state`. Testirao sam funkcije kao `group()`, `count()`, `sort()`, i `limit()` kako bih razumio kako agregirati podatke za Grafana panele.

Utorak, 14. travnja

Ukratko

- Konfiguracija Grafana datasourcea za InfluxDB
- Debugging autentifikacije i connection stringova
- Kreiranje novog dashboarda

Grafana je već bila konfigurirana putem `grafana/provisioning/datasources/influxdb.yaml` no nisam je testirao do danas. Prilikom prvog pokušaja dodavanja panela, dobio sam grešku "unauthorized" što je značilo da kredencijali nisu prošli. Problem je bio što sam u YAML-u koristio environment varijable (`${INFLUXDB_ADMIN_USER}`) koje nisu bile dostupne Grafana containeru. Bar sam tako mislio dok nisam vidio da imam zatipak u `.env` datoteci koji sam onda odmah ispravio. Nakon toga "Test & Save" je bio uspješan. InfluxDB je bio dostupan na `http://influxdb:8086` iz Grafana containera zahvaljujući Docker networku. Query language sam postavio na Flux (ne InfluxQL). Kreirao sam novi dashboard pod nazivom "Zeek Network Monitoring" i spremio ga u provisioning direktorij.

Ponedjeljak, 13. travnja

Ukratko

- Istraživanje Grafana dashboard best practices
- Planiranje panela za prikaz podataka
- Sketching dashboard layouta

Prije implementacije, htio sam planirati koje metrike su najvažnije za nadzor lokalne mreže. Istraživao sam postojeće Zeek/Grafana dashboarde na GitHub-u i Grafana Labs stranici kako bih vidio što drugi monitoring sustavi prikazuju. Odlučio sam fokusirati se na pet ključnih područja:

- promet i bandwidth (konekcije po vremenu, protokoli)

- top komunikatori (najaktivniji source/destination IP-ovi)
- sigurnosni eventi (Zeek notice zapisi, port scanovi, SSH brute-force)
- DNS upit analiza (budući work)
- sumnjive konekcije (`conn_state != "SF"`, neobični portovi)

Za ovaj tjedan cilj je implementirati samo prva tri područja - ostatak dolazi u idućim iteracijama. Napravio sam skicu layouta na papiru s pozicijama panela. Izvori koje sam koristio su Grafana dokumentacija i Corelight ECS-Zeek dashboards repozitorij.

Nedjelja, 12. travnja

Ukratko

- Učenje o Flex skriptnom jeziku
- Uređivanje bilješki - `biljeske.tex`

 Zbog same duljine ovoga dokumenta, dodao sam dodatne *anchore* na tjedne i Sadržaj kako bih se lakše snalazio u dokumentu. Za davanje upita nad InfluxDB-om i Grafanom potrebno je znati Flux jezik. Prethodno sam ga istraživao i trebao bi biti lakši od klasičnog SQL-a. Za izvore sam koristio:

- Get started with Flux
- Flux query language and Influxdb basics
- InfluxDB: Flux Basics
- Intro to Flux Functions
- Samantha Wang — Transform Your Data Using Telegraf & Flux — InfluxDays
- InfluxDB: Intro to Telegraf

Linkovi se mogu naći u `.tex` datoteci te su svi dostupni na WebArchive stranici - provjereno zadnje 15.4.2026 Primjer flux upita:

```
from(bucket: "example-bucket")
  |> range(start: -1d)
  |> filter(fn: (r) => r._measurement == "example-measurement")
  |> mean()
  |> yield(name: "_results")
```

Bucket smo postavili još kad smo postavljali `.env` datoteku i `docker-compose.yml`. Zove se `zeek_logs` te iz nje dohvaćamo zapise.

Subota, 11. travnja

Ukratko

- Provjera podataka u InfluxDB
- Ispravak konfiguracija

- - - - -

Iako se početno nijedan alat nije žalio sa `docker logs [alat] --tail 10` podatci se nisu prebacivali. Pogreška je bila što nisam "izbjegao" . u konfiguraciji Telegrafa i onda je krivo parsirao informacije iz Zeeka. Uz to (što će ostati misterij) Zeek `@scan` politika je zastarijela no ne shvaćam kako kada je verzija tvrdo postavljena (8.0.6). Kako nisam nigdje našao dobru zamjenu linija je u `zeek/local.zeek` samo zakomentirana. Nakon svih promjena podatci su vidljivi u InfluxDB.

Tjedan 6

(4. travnja – 10. travnja 2026.)

Petak, 10. travnja

Ukratko

- pregled stanja M2 milestona i priprema za M3
- potvrđen rad cijelog pipeline-a: Zeek → Telegraf → InfluxDB

Kako je tjedan bližio kraju napravio sam pregled svega što je napravljeno u sklopu M2 milestona. Pokretanjem `sudo ./dipl.sh status` potvrđeno je da su svi servisi aktivni, a pregledom InfluxDB Data Explorera vidljiva su oba measurementa: `zeek_conn` i `zeek_notice`. `zeek_notice` measurement sada prima zapise svaki put kada Zeek detektira nešto što zadovoljava uvjete neke od učitanih policy skripti - port scan ili SSH brute force pokušaj. M2 milestone (rok: 15. travnja) je praktički završen tjedan ranije od planiranog što pruža dobar temelj za nadolazeći M3 koji uključuje izradu Grafana dashboarda. Sljedeći koraci uključuju dizajn dashboarda koji će vizualizirati podatke iz oba measurementa te integraciju `notice.log` upozorenja kao zasebnog panela za sigurnosne događaje.

Četvrtak, 9. travnja

Ukratko

- dodavanje `zeek_notice` measurementa u Telegraf konfiguraciju
- provjera primitka `notice.log` zapisa u InfluxDB-u

Nakon što je u srijeda potvrđeno da Zeek ispravno detektira i bilježi sigurnosne događaje u `notice.log`, sljedeći logičan korak bio je proširiti Telegraf konfiguraciju da šalje i te podatke u InfluxDB, analogno postojećem `zeek_conn` measurementu. Dodan je drugi `[[inputs.tail]]` blok u `telegraf/telegraf.conf` koji čita `/zeek-logs/notice.log` i šalje zapise pod measure `zeek_notice`. Kao tagovi definirani su `note` (tip notifikatora, npr. `Scan::Port_Scan` ili `SSH::Password_Guessing`) i `src_ip`, a kao polje pohranjena je `msg` poruka koju Zeek generira uz svaki `notice`. Nakon restarta Telegrafa naredbom `docker compose restart telegraf` i ponovljenog port scana, u InfluxDB Data Exploreru potvrđeni su novi zapisi pod `zeek_notice` measurementom što znači da cijeli detekcijski lanac radi ispravno.

Srijeda, 8. travnja

Ukratko

- testiranje Zeek policy skripti simulacijom port scana alatom `nmap`
- potvrđen nastanak `notice.log` i ispravna detekcija

Kao što je opisano u utorak, u `zeek/local.zeek` dodane su policy skripte za detekciju skeniranja i SSH brute force napada. Kako bi se potvrdilo da skripte stvarno rade, simuliran je port scan s laptopa prema mobitelu spojenom na AP koristeći `nmap`:

- `nmap -sS 10.0.1.10`

nmap šalje SYN pakete na niz portova - upravo one konekcije koje u `conn.log` ostavljaju trag kao `conn_state: S0` (SYN bez odgovora), što je obrazac koji `policy/misc/scan.zeek` prati. Nakon scana, pregledom `zeek-logs/` direktorija potvrđen je nastanak `notice.log` datoteke - što znači da Zeek nije samo zabilježio promet već ga je i interpretirao kao potencijalno maliciozan. Sadržaj `notice.log` sadržavao je zapise tipa `Scan::Port_Scan` s izvorišnom adresom laptopa i porukom o broju skeniranih portova, što je direktna potvrda da `policy` skripta radi ispravno u našem okruženju.

Utorak, 7. travnja

Ukratko

- proširenje `zeek/local.zeek` s `policy` skriptama za detekciju napada
- istraživanje kako skripte rade iznutra

Na temelju istraživanja provedenog u petom tjednu o `conn_state` poljima i obrascima napada, te neuspješnog pokušaja ručne interpretacije logova Python skriptom (opisano u suboti), zaključeno je da je bolje koristiti već gotove Zeek `policy` skripte koje su dio standardne instalacije unutar kontejnera, nego pisati vlastitu heuristiku. Dostupne skripte provjerene su naredbom:

- `docker exec zeek-ids find /usr/local/zeek/share/zeek/policy -name "*.zeek"`

U `zeek/local.zeek` dodane su dvije `policy` skripte:

- `@load policy/misc/scan-` detektira port scanning praćenjem neuspješnih konekcija (`conn_state: S0, REJ`) prema većem broju portova ili hostova unutar vremenskog prozora; rezultat je `Scan::Port_Scan` ili `Scan::Address_Scan` notice
- `@load policy/protocols/ssh/detect-bruteforcing` - koristi Zeekov `SumStats` framework koji broji neuspješke SSH konekcije po izvorišnom hostu; kada host prijeđe zadani prag (defaultno 30 neuspješnih pokušaja u 30 minuta) generira se `SSH::Password_Guessing` notice

Objekti skripte su dio standardne Zeek instalacije i ne zahtijevaju nikakvo dodatno pisanje koda - samo `@load` direktivu u `local.zeek`. Sve što skripte detektiraju završava u `notice.log` putem Zeekove `NOTICE()` funkcije, što ga čini centralnim mjestom za sigurnosne događaje. Zeek je restartiran kako bi učitao novu konfiguraciju `docker restart zeek-ids`

Subota, 4. travnja

Ukratko

- Neuspješan pokušaj interpretacije logova

Kako bi se povećalo razumijevanje Zeek logova, specifično `conn_state` polja napisana je Python skripta za "ručnu provjeru" logova. Koristeći ulaze u `conn.log`. Zapisi koji dolaze stavljeni su u ugnježdjeni rječnik (eng. *dict*) na način: `{"id.orig_h": {"id.orig_p": {"id.resp_h": {"id.resp_p": [informacije]}}}}` Odnosno prvi ključ je bila adresa izvora, potom

port pa konkatenirana adresa i port odredišta koji za vrijednost imaju listu gdje su svi puni zapisi iz redaka. Nadalje se taj riječnik dijeli na outbound i inbound veze ovisno o prvom ključu tj. izvorišnoj adresi - ako je unutar 10.0.1.0/24 onda spada pod outbound a inače inbound veze. Ovo se nije pokazalo najboljim jer je ovisilo o tome koliko dobro autor napiše heuristiku što nije trivijalan zadatak te se ova ideja i skripta na kraju zanemarila.

Tjedan 5

(28. ožujka – 3. travnja 2026.)

Petak, 3. travnja

Ukratko

- istraživanje Zeek `conn_state` polja kao primarnog indikatora anomalija
- pregled poznatih obrazaca napada vidljivih u `conn.log`

Nakon što je pipeline potvrđen kao funkcionalan, fokus je premješten na razumijevanje kako interpretirati podatke u kontekstu detekcije napada. Zeekovo `conn_state` polje pokazalo se ključnim indikatorom - vrijednosti poput `S0` (SYN poslan, nema odgovora) u velikom broju ukazuju na port scanning, `REJ` (konekcija odbijena) može signalizirati pokušaje pristupa zatvorenim portovima, a `OTH` (promet bez SYN-a) može indicirati presretanje sesija. Proučeni su i obrasci koji su relevantni za outbound detekciju: neuobičajeno veliki `orig_bytes` prema vanjskim IP adresama može ukazivati na eksfiltraciju podataka, dok ponavljajuće konekcije prema istom odredištu na neočekivanim portovima mogu signalizirati C2 (*Command and Control*) komunikaciju. Identificirano je da je za IoC integraciju (M4 milestone) ključno uspoređivati `id.resp_h` polje s poznatim malicioznim IP adresama iz javnih feed-ova. Ovi uvidi poslužiti će kao osnova za dizajn Grafana dashboarda i detekcijskih pravila.

Četvrtak, 2. travnja

Ukratko

- istraživanje najboljih metoda za interpretaciju Zeek logova u sigurnosnom kontekstu
- proučavanje Zeek `notice.log` kao mehanizma za generiranje upozorenja

Istražene su preporučene prakse za rad sa Zeek logovima u kontekstu mrežnog nadzora. Utvrđeno je da `conn.log` sam po sebi nije dovoljan za pouzdanu detekciju napada - potrebno ga je korelirati s ostalim logovima, posebno `dns.log` (sumnjivi DNS upiti, DGA domene) i `ssl.log` (nevažeći certifikati, neuobičajeni cipher suiteovi). Proučen je Zeekov `notice.log` koji agregira upozorenja generirana Zeek skriptama - ovaj log bit će ključan za M4 fazu jer Zeek može nativno detektirati određene obrasce poput port scana ili *brute force* pokušaja putem `@load policy/protocols/ssh/detect-bruteforcing` i sličnih *policy* skripti. Dokumentirani su planovi za proširenje `zeek/local.zeek` s dodatnim politikama za detekciju specifičnih prijetnji, što će biti implementirano u M4 milestoneu.

Srijeda, 1. travnja

Ukratko

- potvrđen primitak podataka u Influxu - vidljivi `zeek_conn` zapisi
- provjera ispravnosti oznaka i polja u Influx data exploreru

Nakon ispravaka Telegraf konfiguracije, pristupljeno je Influx web sučelju na `localhost:8086` radi provjere da podaci stvarno dolaze. U *Data explorer* sučelju potvrđeni su zapisi

pod mjerenjem `zeek_conn` unutar bucketa `zeek-logs`. Provjerena je ispravnost oznaka: `src_ip`, `dst_ip`, `proto`, `service` i `conn_state` vidljivi su kao očekivani tagovi, dok su `src_bytes`, `dst_bytes`, `duration` i `uid` ispravno pohranjena kao `int` i `string` polja. Uočeno je da zapisi s `optional = true` poljima (poput `duration` i `service`) nisu uvijek prisutni, što je očekivano ponašanje jer Zeek ne popunjava sva polja za svaku konekciju - npr. UDP konekcije često nemaju `duration`. Tok podataka Zeek → Telegraf → Influx funkcioniра.

Utorak, 31. ožujka

Ukratko

- debugiranje Telegraf konfiguracije - podaci nisu stizali u InfluxDB
- utvrđen uzrok: neispravna putanja do `conn.log` unutar kontejnera

Pri prvom pokretanju Telegrafa podaci nisu stizali u Influx. Pregledom logova naredbom `docker logs telegraf -f` utvrđeno je da Telegraf nije pronalazio `conn.log` datoteku - ispisivao je upozorenje da datoteka ne postoji ili još nije kreirana. Uzrok je bila neispravna putanja: u `telegraf.conf` putanja je bila postavljena na `/zeek-logs/conn.log`, no `conn.log` Zeek ne kreira odmah pri pokretanju - datoteka nastaje tek kada Zeek zabilježi prvu konekciju i zapiše prvi red u `conn.log`. Dodatno, utvrđeno je da Zeek u standalone načinu rada (`zeek -C -i wlan1 local`) ne rotira logove automatski kao `zeekctl`, već piše kontinuirano u istu datoteku, što je zapravo povoljno za `inputs.tail` plugin koji prati kraj datoteke (nešto kao `tail` kao na Linuxu). Nakon ponovnog pokretanja okoline s aktivnim mrežnim prometom, Telegraf je uspješno počeo čitati `conn.log` i slati zapise prema Influxu.

Ponedjeljak, 30. ožujka

Ukratko

- prvo pokretanje Telegrafa - provjera konfiguracije i veze s Influxom
- uočeni problemi s dozvolama na `zeek-logs/` putanji

Telegraf je pokrenut `docker compose up -d telegraf` te su logovi nadgledani s `docker logs telegraf -f`. Telegraf je učitao konfiguraciju iz `telegraf/telegraf.conf` i uspostavio vezu prema InfluxDB-u na `http://influxdb:8086` koristeći nasumični token iz `.env` datoteke. Uočen je problem s dozvolama: `zeek-logs/` direktorij montiran je u Telegraf kontejner kao `:ro` (read-only), no Telegraf je ispisivao upozorenja vezana uz praćenje datoteke (*inotify*). Problem je riješen provjerom da je putanja dostupna svim korisnicima (`chmod 755 zeek-logs/`) te ponovnim pokretanjem. Osnovna komunikacija između Telegrafa i InfluxDB-a potvrđena je, no obrada podataka još nije verificirana jer je potreban aktivan Zeek s prometom što nije bilo moguće zbog manjka interneta od strane operatera (kasnije popravljeno - modem je upao u neki boot loop pa ga je bilo potrebno ponovno pokrenuti).

Nedjelja, 29. ožujka

Ukratko

- Povezivanje Zeek -> Telegraf -> InfluxDB

Primjetio sam da sam daleko ispred rokova kojih sam si zadao (**M2: Implementacija senzora**) čiji rok je do 15. travnja je praktički napravljen; dakako bolje da je tako nego da kasnim no također ta spoznaja uvelike olakšava rad zbog manjka stresa prema radu. Kao što je jučer navedeno povezao sam Telegraf sa Zeekom i InfluxDB-om. Razlog zašto je trajalo cijeli dan je ukratko Docker, tj. moja obazrivost. Nakon debugiranja putanja i argumenata tj. svojstava međusobni su sustavi povezani.

Subota, 28. ožujka

Ukratko

- *sanity check* cijelog projekta
- Dodavanje Telegrafa u `docker-compose.yml`

Danas nije bilo moguće napraviti puno konkretnog pomaka kako nisam bio i svom uobičajenom testnom okruženju (Zagreb), već sam putovao u Rijeku. Ondje imam slične uvjete te će konkretna implementacija biti nastavljena. Prije puta sam potvrdio rad trenutne okoline. Pripremam se za dodavanje Telegraf agenta (nadalje samo Telegraf) koji bi trebao biti posrednik između Zeeka i InfluxDB. Da rezimiram (*sanity check* za budućnost): nisam mogao direktno spojiti Zeek sa InfluxDB jer Influx ne može čitati direktno Zeek logove (bez obzira na oblik), a Zeek nema mogućnost dodavanja u Influx te da izbjegnem pisanje vlastitog *bridgea* između Zeeka i Influxa otkrio sam da Telegraf radi ono što meni treba. Također Grafana ne može primiti iste logove niti direktno iz Zeeka niti sa Telegrafa pa zato moram imati InfluxDB između jer ju Grafana podržava kao izvor informacija (*data-source*).

Kako nisam u mogućnosti testirati okolinu samo sam izmijenio `docker-compose.yml` da uključuje i Telegraf. Sutra ću dodatno povezati Zeek i Telegraf te Telegraf i Influx a onda mi samo manje više preostaje izrada "instrument ploče" (*dashboarda*) na Grafani.

Tjedan 4

(21. ožujka – 27. ožujka 2026.)

Petak, 27. ožujka

TL:DR

- istraživanje Telegraf agenta - arhitektura i konfiguracija za ingestion Zeek logova u InfluxDB

Nakon što je kompletna Docker okolina potvrđena kao funkcionalna i AP promet uspješno hvatan Zeekom, fokus je premješten na planiranje M2 milestona čija je srž implementacija Telegraf agenta. Telegraf je *plugin-driven* agent otvorenog koda koji služi kao posrednik između izvora podataka i baza podataka - u našem slučaju čita Zeek logove iz `./zeek-logs` / direktorija i šalje ih u InfluxDB. Proučena je službena dokumentacija za `telegraf` Docker image s naglaskom na dva ključna plugina: `inputs.tail` (čitavanje log datoteka u realnom vremenu) i `outputs.influxdb_v2` (slanje podataka u InfluxDB 2.x putem HTTP API-ja). Budući da Zeek logove zapisuje u JSON formatu (što je konfigurirano u `zeek/local.zeek` prošlog tjedna), `inputs.tail` plugin može parsirati svaki redak kao zaseban JSON objekt bez dodatne transformacije. Napravljen je nacrt `telegraf.conf` konfiguracije koja će biti implementirana u sklopu M2.

Četvrtak, 26. ožujka

TL:DR

- UFW firewall blokirao DHCP promet na `wlan1` - dijagnostika i rješavanje
- mobitel uspješno dobiva IP adresu putem DHCP-a

Unatoč tome što su `hostapd` i `dnsmasq` servisi bili aktivni i `wlan1` ispravno konfiguriran s adresom `10.0.1.1/24`, klijentski uređaji nisu mogli dobiti IP adresu. Dijagnostika je započeta pregledom `nft list ruleset` gdje je utvrđeno da UFW (*Uncomplicated Firewall*) presreće DHCP pakete (port 67/UDP) na `ufw-after-input` chainu i prosljeđuje ih u `ufw-skip-to-policy-input` koji ih droppa - što je potvrđeno brojačem od 49 uhvaćenih paketa. Problem je riješen dodavanjem eksplicitnih UFW pravila:

- `ufw allow in on wlan1 to any port 67 proto udp`
- `ufw allow in on wlan1 to any port 68 proto udp`
- `ufw route allow in on wlan1 out on enp2s0`

Nakon primjene pravila, `journalctl -u dnsmasq -f` potvrdio je primitak DHCPDISCOVER paketa i uspješnu dodjelu IP adrese klijentskom uređaju iz raspona `10.0.1.10 - 10.0.1.100`. UFW pravila su dokumentirana radi reproducibilnosti - u budućnosti ih treba ugraditi u `net_setReset_rules.sh` skriptu.

Srijeda, 25. ožujka

TL:DR

- otklonjena greška u `hostapd.conf` - nevažeći znakovi u SSID-u

- dnsmasq konflikt s systemd-resolved na portu 53 - riješeno s bind-interfaces

Pokretanjem `sudo ./dipl.sh start` otkrivene su dvije greške koje su sprječavale podizanje AP okoline. Prva greška bila je u `/etc/hostapd/hostapd.conf` - `hostapd` je odbio pokrenuti se s porukom `Line 6: invalid SSID`. Uzrok su bile zagrade u nazivu mreže (`Dipl_test (NE SPAJATI SE/DO NOT CONNECT)`) koje `hostapd` ne prihvaća kao valjane znakove u SSID stringu. SSID je preimenovan u `Dipl_test_NO_CONNECT`. Druga greška bila je kod `dnsmasq` servisa koji nije mogao kreirati socket na portu 53 (`Address already in use`). Pregledom `ss -tulpn` utvrđeno je da `systemd-resolved` drži port 53 na `127.0.0.53` i `127.0.0.54`. Budući da `dnsmasq.conf` već sadrži `interface=wlan1`, dovoljno je bilo odcommentirati direktivu `bind-interfaces` u `/etc/dnsmasq.conf` čime `dnsmasq` sluša isključivo na navedenom sučelju umjesto na `0.0.0.0`, čime je konflikt s `systemd-resolved` otklonjen bez gašenja sistemskog DNS resolvera.

Utorak, 24. ožujka

TL:DR

- popravak `docker-compose.yml` - neispravna sintaksa `volumes` bloka
- potvrđena funkcionalna Grafana ↔ InfluxDB veza, 3 bucketa pronađena

Validacijom konfiguracije naredbom `docker compose config` otkrivena je sintaksna greška: globalni `volumes`: blok na dnu datoteke sadržavao je `bind mount` sintaksu (`- ./influx -data:/var/lib/influxdb2`) koja je valjana jedino unutar definicije pojedinog servisa, a ne kao samostalni top-level blok. Greška je ispravljena premještanjem `bind mount`ova direktno u `volumes`: sekcije odgovarajućih servisa (`influxdb` i `grafana`), čime su lokalni direktoriji `influx-data/` i `grafana-storage/` ispravno montirani. Nakon ispravka, `docker compose pull` povukao je sve imageove, pri čemu je utvrđeno da `zeekuritey/zeek` repozitorij više ne postoji te je zamijenjen ispravnim `zeek/zeek:8.0.6`. Pokretanjem `docker compose up -d influxdb grafana` (Zeek izostavljen jer zahtijeva `wlan1`) potvrđena je ispravna komunikacija između Grafane i InfluxDB-a kroz Grafanin *Test data source* mehanizam - pronađena su 3 bucketa, što potvrđuje da provisioning putem `grafana/provisioning/datasources/influxdb.yaml` funkcionira ispravno.

Ponedjeljak, 23. ožujka

TL:DR

- prvo pokretanje Zeeka s adapterom - potvrđen capture prometa na `wlan1`
- provjera JSON log outputa u `zeek-logs/`

S dostupnim TP-LINK TL-WN722N adapterom, pokrenuta je kompletna okolina putem `sudo ./dipl.sh start`. Zeek kontejner uspješno je pokrenut s `network_mode: host` i direktnim pristupom sučelju `wlan1`. Pregledom `zeek-logs/` direktorija potvrđen je nastanak očekivanih log datoteka: `conn.log`, `dns.log`, `http.log`, `ssl.log` i ostalih. Inspekcijom

`conn.log` potvrđen je JSON format zapisa, što je rezultat `@load policy/tuning/json-logs` direktive u `zeek/local.zeek`. Svaki zapis sadržava polja poput `ts`, `uid`, `id.orig_h`, `id.resp_h`, `proto`, `service` i `duration` koji će biti ključni za vizualizaciju u Grafani. Zeek logovi potvrđeni su kao ispravan ulazni format za Telegraf `inputs.tail` plugin koji dolazi u M2.

Nedjelja, 22. ožujka

TL:DR

- nadogradnja postojeće `dip1.sh` skripte (obrada grešaka i ispis stanja)
- dodjeljivanje privilegija i postavljanje aliasa za pokretanje

Postojeća upravljačka skripta `dip1.sh` je nadograđena kako bi se olakšalo praćenje servisa i spriječilo ostavljanje sustava u nedefiniranom stanju. Uvedena je obrada grešaka pri pokretanju: ako podizanje Docker kontejnera ne uspije, skripta sada automatski poziva `net_setReset_rules.sh reset`. Time se osigurava da pristupna točka (AP) ne ostane aktivna ako Zeek IDS ne nadzire promet. Proširena je naredba `status`. Umjesto ručne provjere svakog alata zasebno, skripta sada u jednom terminalskom ispisu pregledno prikazuje stanje `systemd` procesa (`hostapd`, `dnsmasq`) i svih Docker kontejnera. Kako bi se ubrzao svakodnevni rad, skripti su dodijeljene izvršne privilegije (`chmod +x`) te je u konfiguraciju ljuste dodan lokalni alias koji omogućava njeno pokretanje iz bilo kojeg direktorija.

Subota, 21. ožujka

TL:DR

- pregled stanja projekta na kraju M1 milestona i planiranje tjedna

Napravljen je pregled cjelokupnog stanja projekta na kraju M1 milestona. Struktura repozitorija sadrži sve očekivane komponente: `net_setReset_rules.sh`, `docker-compose.yml`, `zeek/local.zeek`, `grafana/provisioning/datasources/influxdb.yml` i `.env` datoteku s kredencijalima. Identificirani su otvoreni zadaci za tekući tjedan: testiranje kompletne okoline s fizičkim adapterom, verifikacija Zeek JSON log outputa i priprema za M2 koji uključuje implementaciju Telegraf ingestion pipeline-a. Pregledom `docker-compose.yml` uočena je potencijalna sintaksna greška u `volumes` bloku koja će biti adresirana idući radni dan.

Tjedan 3

(14. ožujka – 20. ožujka 2026.)

Petak, 20. ožujka

TL:DR

- izrada `dipl.sh` - skripta za jednokratno pokretanje kompletne okoline

Budući da okolina sada obuhvaća više komponenti (AP, Zeek, InfluxDB, Grafana), javila se potreba za jedinstvenom skriptom koja ih sve pokreće i gasi redom. Napravljena je `dipl.sh` skripta koja sekvencijalno poziva `net_setReset_rules.sh set` za postavljanje AP okoline, a zatim pokreće sve Docker servise putem `docker compose up -d`. U slučaju greške pri pokretanju Dockera, skripta automatski poziva `reset` AP-a kako sustav ne bi ostao u nedefiniranom stanju. Podržane su tri naredbe: `start`, `stop` i `status`, gdje `status` prikazuje stanje i AP servisa i Docker kontejnera na jednom mjestu. Tjedan je zaključen s kompletnom Docker okolinom spremnom za daljnji razvoj u sklopu M2 milestona.

Četvrtak, 19. ožujka

TL:DR

- testiranje pokretanja ažuriranog `docker-compose.yml` - uočeni manji konfiguracijski problemi

Pokrenuta je nova konfiguracija naredbom `docker compose up -d` te su praćeni logovi svakog kontejnera zasebno. Zeek i InfluxDB pokrenuli su se bez problema, dok je Grafana inicijalno imala problem s dozvolama na volume direktoriju. Problem je riješen ispravkom vlasništva direktorija na host sustavu. Provjerena je dostupnost Grafana sučelja na `localhost:3000` i uspješno obavljena prijava s defaultnim kredencijalima. Konfiguracija InfluxDB data sourcea u Grafani ostavljena je za sljedeći dan.

Srijeda, 18. ožujka

TL:DR

- ažuriran `docker-compose.yml` - dodani InfluxDB i Grafana servisi

Postojeći `docker-compose.yml` (koji je sadržavao samo Zeek servis) proširen je s InfluxDB i Grafana servisima. Definirana je interna Docker bridge mreža kako bi servisi mogli međusobno komunicirati, te su postavljeni volumeni za persistiranje podataka. Postavljene su environment varijable za inicijalizaciju InfluxDB instance (`bucket`, `org`, `token`). Grafana je konfigurirana da sluša na portu 3000, a InfluxDB na 8086. Zeek zadržava `network_mode: host` konfiguraciju zbog potrebe za direktnim pristupom mrežnom sučelju `wlan1`.

Utorak, 17. ožujka

TL:DR

- istraživanje Grafana Docker konfiguracije i načina spajanja na InfluxDB

Istražena je konfiguracija Grafane unutar Dockera, uključujući persistiranje dashboarda i konfiguracije putem volumena. Proučeno je kako Grafana komunicira s InfluxDB-om koristeći Flux query jezik. Posebna pažnja posvećena je provisioningu data sourcea putem YAML datoteka kako bi se izbjeglo ručno konfiguriranje pri svakom pokretanju kontejnera. Napravljen je grubi plan strukture proširenog `docker-compose.yml` koji će obuhvatiti sve komponente: Zeek, InfluxDB i Grafana.

Ponedjeljak, 16. ožujka

TL:DR

- istraživanje InfluxDB Docker konfiguracije i sheme pohrane Zeek logova

Fokus dana bio je na InfluxDB-u i njegovoj konfiguraciji unutar Docker okoline. Proučena je službena dokumentacija za `influxdb` Docker image, s naglaskom na inicijalizaciju baze i postavljanje autentikacijskih tokena putem varijabli okruženja. Razmatrano je kako strukturirati Zeek logove za unos u InfluxDB - konkretno koji su logovi relevantni (`conn.log`, `dns.log`, `http.log`). Zeek logove u JSON formatu moguće je direktno ubaciti u InfluxDB, što je potvrđeno pregledom dokumentacije. Detaljna implementacija pipeline-a planirana je za M2 milestone.

Nedjelja, 15. ožujka

TL:DR

- istraživanje Docker networking modela za međuservisnu komunikaciju

Istražio sam kako Docker omogućuje komunikaciju između kontejnera putem internih bridge mreža. Posebno me zanimalo kako Zeek, koji koristi `network_mode: host` zbog direktnog pristupa mrežnim sučeljima, može dijeliti logove s ostalim servisima. Utvrđeno je da je rješenje mapiranje Zeek log direktorija kao volumena koji ostali kontejneri mogu čitati. Proučena je i osnovna dokumentacija za `docker-compose` networking kako bi se razumjelo definiranje zajedničke interne mreže između servisa.

Subota, 14. ožujka

TL:DR

- uočen problem s nativnom podrškom za Zeek, Grafanu i InfluxDB na trenutnom OS-u
- odluka o migraciji na Docker

Prilikom pregleda dosadašnje konfiguracije uočene su nestabilnosti pri pokretanju Grafane i InfluxDB-a nativno na CachaOS-u (moj trenutni OS, baziran na Arch Linuxu). Dio paketa nije dostupan direktno u repozitorijima ili ima zastarjele verzije, što otežava daljnji razvoj. Kako Zeek već radi unutar Dockera, logičan sljedeći korak je migracija i preostalih

servisa na isti pristup. Pokretanje svih komponenti unutar Dockera donosi prednosti: jednaka okolina na svim uređajima, lakše upravljanje verzijama i eliminacija dependency konflikata. Iduće dane planirano je istražiti Docker networking model i konfiguracije pojedinih servisa.

Tjedan 2

(7. ožujka – 13. ožujka 2026.)

Petak, 13. ožujka

TL:DR

- izrada `/etc/hostapd/hostapd.conf` datoteke

Izrada `/etc/hostapd/hostapd.conf` u njenoj potpunosti. Ona diktira ponašanje AP-a. Definirani:

- sučelje za koristiti (`wlan1`)
- driver
- ssid (`Dipl_test` (`NE SPAJATI SE/DO NOT CONNECT`))
- kanal (`6`)
- ... ostale stvari poput 802.11n standarda, WPA2 i gašenje izolacije klijenata `ap_isolate=0` tako da Zeek može ubuduće gledati promet.

Četvrtak, 12. ožujka

TL:DR

- pokušaj testiranja `net_setReset_rules.sh` - otkriven propust
- izrada `/etc/dnsmasq.conf` datoteke

Pokušao sam testirati radi li moja skripta i mogu li se spojiti na novonastali AP. Izrada osnovne konfiguracije `dnsmasq.conf` za konfiguraciju DHCP-a (raspon adresa od 10.0.1.10 - 10.0.1.100, sa trajanjem 24 sata) i DNS-a (korišten Google i Cloudflare DNS serveri) Sutra potrebno napraviti `/etc/hostapd/hostapd.conf`.

Srijeda, 11. ožujka

TL:DR

- Instalirana Grafana

Samo sam instalirao Grafanu zbog viška poslovnih i osobnih obveza.

Utorak, 10. ožujka

TL:DR

- preuzet Zeek - dodan `src/docker-compose.yml`
- isprobavanje mogućnosti putem Zeek online stranice

Gledao sam mogućnosti Zeeka i upoznao se sa njihovim skriptnim jezikom. Kako Zeek nije nativno podržan na mojem operacijskom sustavu moram ga pokrenuti preko Dockera za što je napravljena dodatna .yml skripta.

Gledajući unaprijed ovo je dobro ako želimo dodati bazu podataka u koju ćemo spremati logove koje će kasnije Grafana grafički prikazivati.

Potrebno je još testiranja sa jezikom Zeeka

Ponedjeljak, 9. ožujka

TL:DR

- ažurirana *net_setReset_rules.sh*
- dodani "rukohvati" kako se ne bi postavila dupla pravila tj. očistila nepostojeća

Ažurirao sam skriptu sa rukohvatima koristeći **systemctl** kako bi provjerio ako je servis **hostapd** (on po *defaultu* ne radi) pokrenut i na temelju toga dozvoljava da se AP setira odnosno resetira.

Nedjelja, 8. ožujka

TL:DR

- Ažurirana skripta koja vraća laptop i sučelja u izvorno stanje
- *net_setReset_rules.sh* je gotova

Sve što sam napravio jučer je samo trebalo reverzirati.

Subota, 7. ožujka

TL:DR

- Napisan prvi dio skripte *net_setReset_rules.sh*
- Opisana sučelja koja će se koristiti
- Odabran uređaj koji će biti novi AP (odabran zbog dostupnosti)
- Definirana pravila i naredbe

Započeta izrada skripte. Današnji cilj je samo pretvoriti laptop u bridge/switch između žičanog i bežičnog sučelja. Ethernet port se spaja na standardni ruter tj sučelje rutera kojeg obično pruža ISP (*Internet Service Provider*). Dobivanje svih mrežnih sučelja na OS baziranom na Arch Linuxu dobivamo s **ip link** ili **ip a** naredbom. Za svrhe rada zanimljiva će biti samo 2 sučelja: **enp2s0** i **wlan1** (**wlan0** je mrežno sučelje kartice laptopa koja nije dovoljno "jaka" da podrži spajanje više uređaja na nju). **enp2s0** je Ethernet mrežno sučelje dok je **wlan1** sučelje TP-LINK TL-WN722N vanjske mrežne kartice.

Prvi zadatak je sada povezati ta dva sučelja da oni međusobno interno propuštaju sav promet (koji će Zeek kasnije nadgledati).

Započeto sa pregledom Arch Wiki - Software access point te Arch Wiki - Internet sharing članaka. Inače se mora pokrenuti i `iw list` da vidimo podržava li naša kartica AP način rada, no kako sam već radio s njom znam da podržava. Koristiti ćemo `nmcli` kako bi izolirali mrežno sučelje da **NetworkManager** ne bi ugasio AP što je česti problem s kojim sam se prije susreo. Nakon toga potrebno je omogućiti *IP proslijeđivanje* koristeći `ip_forward` i spajajući `enp2s0` i `wlan1`. Za kraj je definiran set `nftables` pravila. Sutra je cilj vratiti `enp2s0` i `wlan1` u prethodno stanje tj. napraviti cijelu radnju reverzibilnom.

Tjedan 1

(27. veljače – 6. ožujka 2026.)

Petak, 6. ožujka 2026.

TLDR

- Istraživanje i razvoj skripte za postavljanje mrežnih pravila
- Tokom vikenda razviti skriptu - pobrinuti se da se vrate postavke na staro kako se vrti na mojem računalu
- skripta: *net_setReset_rules.sh*

Izrada skripte koja olakšava očuvanje i postavljanje mrežnih postavki na računalu. Glavni cilj je osigurati da se nakon testiranja AP okruženja sve vrati na staro bez ručnih intervencija.

Započet razvoj *net_setReset_rules.sh* datoteke. Skripta privremeno isključuje upravljanje USB adapterom unutar sustava (*nmccli*) i postavlja *dnsmasq* i *hostapd* naredbe. Ovim se izbjegava miješanje s postavkama ostatka sustava. Preko vikenda slijedi dovršavanje logike za automatsko brisanje svih pravila i vraćanje mrežnog stacka u početno stanje.

Četvrtak, 5. ožujka 2026.

- Istaživanje grafičkog rješenja za izradu projekta
- Najbolji izbor je čini se Grafana
- Dokumentacija Grafane
 - ...open source community and our growing number of users, Grafana introduces new innovations in every release to improve how you visualize, correlate, and share your data

Istraživanje načina za grafički prikaz mrežnih podataka. Potrebno je odabrati alat koji može jasno prikazati kamo uređaji iz mreže šalju podatke.

Grafana je odabrana kao najbolji izbor zbog jednostavnog povezivanja s bazama podataka i mogućnosti slaganja preglednih ploča (dashboards). Proučena je službena dokumentacija s naglaskom na vizualizaciju tokova podataka u realnom vremenu. Alat omogućuje brzo uočavanje neobičnih veza (anomalija), što je ključno za cilj projekta.

Srijeda, 4. ožujka 2026.

- Razrada rješavanja putem laptopa - potreban adapter za "glumljenje" mrežnog sučelja
- istražiti postavljanje privremenih pravila (kako bi se laptop mogao vratiti u prvotno stanje)

Ideja je pretvoriti laptop u switch/AP tako da u Ethernet port spojim ruter a na laptop spojim dodatni adapter koji glumi WiFi sučelje i na njega spojim ostale uređaje. Na taj način na moj ruter će biti spojen samo laptop, a sve ostalo na laptop (koji se kasnije može zamijeniti)

Trebalo bi razmisliti kako osigurati prenosivost ovog rješenja kako ne bi ovisilo o uređaju na kojem se vrti

Utorak, 3. ožujka 2026.

- istraživanje o okolini i potencijalnim alatima koje bi trebao koristiti
 - Alati na Linuxu: hostapd, dnsmasq
 - Za instalirati: Zeek, Grafana
 - Zeek (bivši Bro):
 - ...passive, open-source network traffic analyzer
 - ...operators use as network security monitor (NSM) to support investigations of suspicious or malicious activity
 - ...also supports a wide range of traffic analysis tasks beyond the security domain, including performance measurement and troubleshooting
- Tekst preuzet iz dokumentacije - izabran zbog modela otvorenog koda i sveobuhvatnosti alata
- bilo je nekoliko sličnih alata poput Sutricate ili programiranja od nule ali Zeek nudi sve i još je scriptable*
- dnsmasq i hostapd mi trebaju za pretvaranja laptopa u pristpnu točku #TODO još istražiti
 - Uz istraživanje i preinake ovo rješenje se može vrititi na Raspberry PI pločicama - eventualno pri kraju rada

Ne bi loše bilo ovo napraviti sve na Dockeru kako bi se izbjegao nekakav dependency hell tj. sve se sa "jednim gumbom" instalira

Ponedjeljak, 2. ožujka 2026.

- Nadopunjavanje milestonova/ izrada roadmapa
- Osnovno istraživanje o temi i obujmu/scopu rada

Nedjelja, 1. ožujka 2026.

- Kreiranje LaTeX predloška za strukturirano vođenje dnevnih zabilješki.
- Razrada plana rada i raspodjela faza izrade za nadzor mrežnog prometa.

Prvi zapis: Zadatak i plan rada

Tema: Izrada rješenja za nadzor komunikacije lokalne mreže sa naglaskom na izlazne tokove podataka iz mreže

(EN: *Development of communication monitoring solutions within the local network with focus on outbound data streams from network*).

Zadatak: S porastom broja povezanih uređaja u lokalnim mrežama, nadzor mrežnog prometa postao je ključan element za pravovremenu detekciju sigurnosnih prijetnji i neovlaštenih pristupa. Standardna mrežna oprema često ne pruža dovoljnu vidljivost u specifičnosti dolaznih i odlaznih mrežnih konekcija, zbog čega je otežano utvrditi s kojim točno vanjskim resursima komuniciraju lokalni uređaji te tko s vanjske mreže pokušava pristupiti lokalnim resursima. U sklopu diplomskog rada potrebno je osmisлити i implementirati mrežnu arhitekturu koja će služiti kao središnja pristupna točka za lokalnu mrežu, omogućavajući usmjeravanje i analizu mrežnog prometa. Potrebno je postaviti i konfigurirati odgovarajuća programska rješenja za inspekciju mrežnih sesija i detekciju upada kako bi se zabilježili komunikacijski obrasci i identificirali potencijalni indikatori kompromitiranosti. Na temelju prikupljenih podataka, potrebno je implementirati grafičko sučelje koje će vizualno prikazivati aktivne uređaje u mreži, smjerove njihove komunikacije i evidentirane sigurnosne događaje. Rješenje treba temeljiti na slobodno dostupnim tehnologijama otvorenog koda.

Milestoneovi (Rok: 15. lipnja 2026.):

M1: Mrežna infrastruktura (do 20. ožujka): Konfiguracija AP okruženja i implementacija izolacije mrežnog prometa te NAT-a pomoću skripti.

M2: Implementacija senzora (do 15. travnja): Instalacija i podešavanje IDS-a za inbound detekciju te alata za ekstrakciju metapodataka i outbound sesija.

M3: Pohrana i vizualizacija (do 5. svibnja): Konfiguracija vremenske baze podataka i spajanje senzora te izrada interaktivnog GUI-ja nadzorne ploče.

M4: Testiranje i IoC integracija (do 20. svibnja): Uvoz indikatora kompromitacije (IoC) i simulacija malicioznog prometa radi potvrde ispravnosti rada detekcijskih mehanizama.

M5: Dokumentacija i predaja (do 15. lipnja): Dokumentiranje koda, arhitekture i rezultata analize te finalna predaja rada.